

1. **Scenario:** A program needs to determine the largest prime factor of a given number for mathematical computations.

Write logic to find the largest prime factor of a given number.

- a. Read the two input
 - b. Compute the gcd first, Check if b is not equal to zero , then assign a value to b and b value will be of a mod b.
 - c. If b is zero then return a value
 - d. Create a variable lcm and calculate $a.b / \text{gcd}(a,b)$
 - e. Return the LCM value
2. **Scenario:** A system needs to convert a Roman numeral string into its integer equivalent for numerical processing.
- Write logic to convert a given Roman numeral string into an integer.
3. **Scenario:** A text-processing application analyzes a list of words to find the longest common prefix.

Write logic to find the longest common prefix among a list of strings.

['we', 'were', wear'], we

- a. Read the input from the user
- b. Find the minimum string that would be the prefix for upcoming words
- c. Iterate through each of the word and check if each character matches if not stop it and move to next word
- d. Return the common prefix

4. **Scenario:** A calculator program takes two numbers and an operator to perform a mathematical operation.

Write logic to take two numbers and an operator (+, -, *, /) and perform the corresponding operation.

- a. Read the numbers and its operators
 - b. Create a function for each operations and write calculation logic
 - c. Create a if-elif-else statement to check operator is valid and if not return "Invalid operator"
 - d. If user given valid operation route to that appropriate function and begin calculation
 - e. Return the calculated result
5. **Scenario:** A lottery system assigns special numbers where Armstrong numbers have a special significance.

Write logic to check whether a given number is an Armstrong number.

>> If a given numbers is equal to n raised to the power of the length of the digits

- a. Read the input
- b. Assign a temp variable to n, armstrong variable to 0
- c. Iterate through loop and find the remainder by taking mod of n
- d. Compute the power of length of digits and Add to the armstrong
- e. Reduce the temp by 10 floor division
- f. If armstrong is equal to n then it is a "Armstrong number " else it is not a Armstrong number?
- g. Return the result

6. **Scenario:** A task management application allows users to add, remove, and view tasks in a to-do list.

Write logic to implement a to-do list where users can add, remove, and view tasks.

- a. **Get the input from the user until they quit**
- b. **Create a list todo**
- c. **Create a function for add remove and view**
- d. **Append the input to the add fuction**
- e. **If remove get the user input index and use pop fuction to remove that particular input from the list,**
- f. **If view function return the list**

7. **Scenario:** A weather application requires a function to convert temperature between Celsius and Fahrenheit.

Write logic to convert a given temperature from Celsius to Fahrenheit and vice versa.

8. **Scenario:** A security system needs to verify whether a given string has all unique characters.

Write logic to check whether all characters in a given string are unique.

- a. **Read the input string**
- b. **Create a set**
- c. **Iterate through each letter from the string**
- d. **At first occurrence add to set.**
- e. **Check if that letter exist in set. If yes then return “string is not unique”**
- f. **If not exist then “string is unique”**

9. **Scenario:** A text analyzer tool identifies the longest word in a given sentence for readability analysis.

Write logic to find the longest word in a given sentence.

['programming is every where']

- a. **Read the input**
 - b. **Create a dictionary and for each word calculate the count of each word in a given sentence. Store each key is word and count is the value**
 - c. **Iterate through dict check max count store it in a variable and after comparing with all keys**
 - d. **Return the max count key**
10. **Scenario:** A password manager generates secure passwords based on user-defined length and complexity.
- Write logic to generate a random password of a given length, containing uppercase, lowercase, digits, and special characters.
- a. **Import the random module**
 - b. **Declare uppercase , lowercase, digit and special character variable**
 - c. **Get the length from the user**
 - d. **Use the random function to generate random string.**
 - e. **Return the string**